

```
!pip -q install d2l==0.17.0
```

```
_____ 83.2/83.2 kB 8.4 MB/s eta 0:00:00
_____ 12.4/12.4 MB 140.8 MB/s eta 0:00:
_____ 76.7/76.7 kB 9.0 MB/s eta 0:00:00
_____ 59.8/59.8 kB 6.1 MB/s eta 0:00:00
_____ 1.6/1.6 MB 89.1 MB/s eta 0:00:00
```

```
import tensorflow as tf #
from d2l import tensorflow as d2l # вспомогательные функции для данных, обуч
# AlexNet
def net():
    return tf.keras.models.Sequential([ # контейнер
        # большое ядро 11 x 11 для больших объектов, но сдвиг 4, чтобы существенно
        tf.keras.layers.Conv2D(filters=96, kernel_size=11, strides=4,
            activation='relu'), # первый сверточный слой
        tf.keras.layers.MaxPool2D(pool_size=3, strides=2), # субдискретизация
        # меньшее ядро 5 x 5, но большее количество фильтров. Набивка, чтобы сохра
        tf.keras.layers.Conv2D(filters=256, kernel_size=5, padding='same',
            activation='relu'), # второй сверточный слой
        tf.keras.layers.MaxPool2D(pool_size=3, strides=2), # субдискретизация
        # сверточные слои с малым ядром 3 x 3, с набивкой и еще большим количество
        tf.keras.layers.Conv2D(filters=384, kernel_size=3, padding='same',
            activation='relu'),
        tf.keras.layers.Conv2D(filters=384, kernel_size=3, padding='same',
            activation='relu'),
        # последний сверточный слой с 256 фильтрами, малым ядром 3 x 3.
        tf.keras.layers.Conv2D(filters=256, kernel_size=3, padding='same',
            activation='relu'),
        tf.keras.layers.MaxPool2D(pool_size=3, strides=2), # субдискретизация
        tf.keras.layers.Flatten(), # слой, который вытягивает входной массив в век
        # полносвязный слой
        tf.keras.layers.Dense(4096, activation='relu'),
        tf.keras.layers.Dropout(0.5), # для лучшего обобщения
        # полносвязный слой
        tf.keras.layers.Dense(4096, activation='relu'),
        tf.keras.layers.Dropout(0.5), # для лучшего обобщения
        # последний полносвязный слой с числом нейронов по количеству классов. Для
        tf.keras.layers.Dense(10)
    ])
```

```
X = tf.random.uniform((1, 224, 224, 3)) # вход (случайно)
for layer in net().layers:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
Conv2D output shape:      (1, 54, 54, 96)
MaxPooling2D output shape: (1, 26, 26, 96)
Conv2D output shape:      (1, 26, 26, 256)
MaxPooling2D output shape: (1, 12, 12, 256)
Conv2D output shape:      (1, 12, 12, 384)
```

```

Conv2D output shape: (1, 12, 12, 384)
Conv2D output shape: (1, 12, 12, 256)
MaxPooling2D output shape: (1, 5, 5, 256)
Flatten output shape: (1, 6400)
Dense output shape: (1, 4096)
Dropout output shape: (1, 4096)
Dense output shape: (1, 4096)
Dropout output shape: (1, 4096)
Dense output shape: (1, 10)

```

```

input_shape=(None,224, 224, 3)
nn=net()
nn.build(input_shape)
nn.summary()

```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_11 (Conv2D)	(None, 54, 54, 96)	34,944
max_pooling2d_9 (MaxPooling2D)	(None, 26, 26, 96)	0
conv2d_12 (Conv2D)	(None, 26, 26, 256)	614,656
max_pooling2d_10 (MaxPooling2D)	(None, 12, 12, 256)	0
conv2d_13 (Conv2D)	(None, 12, 12, 384)	885,120
conv2d_14 (Conv2D)	(None, 12, 12, 384)	1,327,488
conv2d_15 (Conv2D)	(None, 12, 12, 256)	884,992
max_pooling2d_11 (MaxPooling2D)	(None, 5, 5, 256)	0
flatten_3 (Flatten)	(None, 6400)	0
dense_7 (Dense)	(None, 4096)	26,218,496
dropout_5 (Dropout)	(None, 4096)	0
dense_8 (Dense)	(None, 4096)	16,781,312
dropout_6 (Dropout)	(None, 4096)	0
dense_9 (Dense)	(None, 10)	40,970

Total params: 46,787,978 (178.48 MB)
Trainable params: 46,787,978 (178.48 MB)

```

batch_size = 128 # размер пакета
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size, resize=224)

```

```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
26421880/26421880 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
5148/5148 ————— 0s 1us/step

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-data/4422102/4422102> — 0s 0us/step

%%time

lr, num_epochs = 0.01, 10 # шаг обучения, число эпох.

d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.try_gpu())

loss 0.328, train acc 0.880, test acc 0.875

1406.2 examples/sec on /GPU:0

CPU times: user 6min 40s, sys: 22.5 s, total: 7min 3s

Wall time: 7min 50s

<Sequential name=sequential_4, built=True>

